Insurance Platform

Architektur-Roadmap und technisches Umsetzungs-Backlog

Dokumenttyp: Lebende Architektur-Roadmap und priorisiertes Engineering-Backlog **Zuletzt aktualisiert:** 28. Oktober 2026 **Geltungsbereich:** Insurance-Platform-Repository und hier dokumentierter Arbeitsstand **Status:** Planungsgrundlage – die offenen Punkte sind keine Aufforderung, sie sofort umzusetzen

> **Wichtig für die aktuelle Lernphase:** Zuerst den vorhandenen Code und die Abläufe verstehen. Nichts in diesem Dokument muss jetzt umgesetzt werden. Dieses Backlog bewahrt offene Architektur- und Qualitätsfragen, damit sie nach der Einarbeitung gemeinsam priorisiert werden können.

1. Zweck und Pflege

Dieses Dokument hält fest, was die Plattform später für einen verlässlichen produktiven Betrieb und eine gut erweiterbare Architektur noch benötigt. Es verbindet drei übliche Engineering-Artefakte:

- eine **Architektur-Roadmap** für die sinnvolle Reihenfolge größerer Schritte,
- ein **Technical-Debt-/Risiko-Register** mit konkreten offenen Punkten,
- einen Verweis auf **Architecture Decision Records (ADRs)**, sobald eine wesentliche Entwurfsentscheidung getroffen wird.

Es ist kein Sprintplan und enthält keine erfundenen Termine. Die Markdown-Datei ist die **maßgebliche, fortlaufend zu pflegende Fassung**; das PDF ist ein lesbarer Export. Bei jeder wesentlichen Änderung sollten beide Fassungen aktualisiert werden.

1.1 Statuswerte

Status	Bedeutung
Offen – erst verstehen	Code und Auswirkung nachvollziehen; noch keine Änderung beginnen
Bereit	Problem verstanden, Umfang und Abnahmekriterien geklärt
In Arbeit	Änderung wird umgesetzt
Erledigt	Abnahmekriterien erfüllt, passende Tests/Dokumentation aktualisiert
Zurückgestellt	Bewusst nicht umgesetzt; Grund dokumentieren
Nicht erforderlich	Nach Prüfung kein aktueller Bedarf; Begründung festhalten

Ein Punkt wird nicht allein dadurch **Erledigt**, dass der Code geändert wurde. Die unten genannten Abnahmekriterien und gegebenenfalls Tests müssen ebenfalls erfüllt sein.

1.2 Umgang mit dem Backlog

1. Vor Beginn eines Punkts zuerst die angegebenen Codepfade und verwandten Tests lesen.
2. Prüfen, ob der Punkt durch inzwischen erfolgte Änderungen teilweise oder ganz überholt ist.
3. Umfang und Abnahmekriterien an die tatsächlichen Produktanforderungen anpassen; Sicherheits- und Datenschutzentscheidungen nicht erraten.
4. Ein größeres Architekturthema vor der Implementierung als ADR festhalten, wenn mehrere ernsthafte Lösungswege bestehen.
5. Änderungen klein und überprüfbar halten; passenden Test ergänzen und die Dokumentation gemeinsam mit dem Code aktualisieren.
6. Status, Ergebnis, Datum und gegebenenfalls Folgeschritt in diesem Dokument pflegen. Einen erledigten Punkt nicht kommentarlos löschen.

2. Architekturleitplanken

Diese Leitplanken sind die derzeit sinnvolle Ausgangshypothese. Eine grundlegende Neuarchitektur ist **nicht** vorgesehen, solange Anforderungen keinen konkreten Anlass dafür liefern.

- **Modularer Monolith beibehalten.** Fachmodule in einem deploybaren Spring-Boot-System sind derzeit einfacher zu entwickeln und zu betreiben als vorzeitig getrennte Microservices.
- **Zwei passende Kommunikationsarten bewusst verwenden:**

  ```
  Sofort benötigte Antwort: synchroner Modulvertrag (XxxApi)
  Bereits eingetretenes Ereignis: Domain Event --> reagierende Module
  ```

  Beispiele im Code: Claim fragt Deckung synchron über `PolicyApi` ab. `PolicyIssuedEvent` löst nachgelagerte Rechnungserstellung aus; `ClaimApprovedEvent` löst nachgelagerte Auszahlungserstellung aus.
- **Modulverträge klein und explizit halten.** Andere Module sollen öffentliche `XxxApi`-Verträge und stabile fachliche Datentransferobjekte verwenden statt fremde Implementierungsdetails oder JPA-Entities zu importieren.
- **Datenbank und Migrationen zuverlässig betreiben.** H2 bleibt bequem für schnelle lokale Feedbackschleifen; PostgreSQL ist die bereits vorgesehene relationale Integrations-/lokale Laufzeit. Nicht ohne konkreten Bedarf eine zweite Produktionsdatenbankfamilie hinzufügen.
- **Verhalten durch Tests absichern.** Geschäftsregeln durch Domain-/Service- Tests, Modulgrenzen durch Architekturtests und Datenbank-/HTTP-Verhalten durch passende Integrationsprüfungen abdecken.
- **Kundendaten konsequent schützen.** Rollenprüfung allein bedeutet nicht, dass ein Benutzer nur seine eigenen Versicherungsdaten sehen darf.
- **Produktionstauglichkeit durch Anforderungen definieren.** Outbox, externe Identitätsplattform, zusätzliche Infrastruktur und komplexe Observability erst dann einführen, wenn Zuverlässigkeits-, Sicherheits- oder Betriebsanforderungen feststehen.

2.1 Zweiebenen-Kommunikation im aktuellen System

```
REST /api
   |
   v
Modularer Monolith
   |-- synchron: Policy --> Customer, Product, Underwriting
   |-- synchron: Claim --> Policy (Snapshot und Deckung am Schadentag)
   |
   |-- PolicyIssuedEvent ------> Billing: Rechnung, falls noch keine existiert
   `-- ClaimApprovedEvent -----> Billing: Auszahlung, falls noch keine existiert
```

Ein Ereignis bedeutet, dass die Folgearbeit nach dem auslösenden Vorgang verarbeitet werden kann. Verbraucher müssen daher mit Wiederholung, Fehlern und einer möglichen Verzögerung umgehen. Das Repository enthält Listener, aber eine dauerhaft gespeicherte Outbox und die gewünschte Wiederherstellung nach Prozessausfall sind nicht als produktiv zugesichert dokumentiert.

3. Priorisierte Roadmap

Die Reihenfolge ist absichtlich **phasenbezogen, nicht datumsbezogen**. Die aktuelle Arbeitsbaumversion kann sich gegenüber dem letzten Commit unterscheiden; vor jedem Schritt muss deshalb der zu diesem Zeitpunkt tatsächlich vorhandene Code geprüft werden.

Phase	Ziel	Ergebnis vor dem nächsten Schritt
0 – Verstehen	Ist-Architektur, Änderungen, fachliche Abläufe und Tests nachvollziehen	Mindestens einen Policy- und einen Claim-Ablauf ohne Vorlage erklären können
1 – Grundlage stabilisieren	PostgreSQL/Flyway, Schema und reproduzierbare Tests in Einklang bringen	Sauber initialisierbare und validierte Datenbank ohne doppelte Flyway-Versionen
2 – Fachabläufe vervollständigen	Claim/Policy/Billing Ende zu Ende testen und Schnittstellen konsistent halten	Fachregeln und Fehlerfälle sind dokumentiert und durch geeignete Tests abgedeckt
3 – Sicherheit und Identität	Authentifizierung und kundengenaue Autorisierung an fachliche Anforderungen anpassen	Kein Benutzer kann nicht zugeordnete Kundendaten lesen oder ändern
4 – Ereigniszuverlässigkeit entscheiden	Benötigte Garantien und Fehlerverhalten für modulübergreifende Events festlegen	Bewusste, getestete Entscheidung für den tatsächlich erforderlichen Zuverlässigkeitsgrad
5 – Frontend und API abgleichen	Login und benötigte Schreib-/Detailabläufe vollständig und sicher unterstützen	UI, DTOs, Rollen und API-Vertrag verhalten sich konsistent
6 – Betrieb vorbereiten	Konfiguration, Startverhalten, Logs, Health und Lieferprozess absichern	Reproduzierbarer Build/Deploy und diagnostizierbarer Dienst für die vereinbarte Zielumgebung
7 – Modulqualität kontinuierlich sichern	Abhängigkeiten, Verantwortlichkeiten und Tests modular halten	Modulgrenzen werden automatisiert geprüft; Änderungen verletzen die Regeln nicht

Phase 0 ist die **jetzige Lernphase**. Phase 1 ist der erste konkrete technische Risikopunkt, sobald du mit der Einarbeitung fertig bist. Phasen 3 und 4 brauchen Entscheidungen über Produkt- und Betriebserwartungen; nicht vorschnell die technisch aufwendigste Lösung wählen.

4. Offene Punkte und Abnahmekriterien

R-01 – Flyway-Versionen und PostgreSQL-Migrationen

Priorität: P1 – vor verlässlicher PostgreSQL-Nutzung **Status:** In Arbeit – Mapping-Regeln festgelegt; PostgreSQL-Integrationstest noch erforderlich **Betroffene Dateien:** `backend/src/main/resources/db/migration/`, `backend/src/main/resources/application.yml`, `backend/src/main/resources/application-local.yml`, `docker-compose.yml`

Beobachtung am 28.10.2026: Die Flyway-Historie reicht von V1 bis V10. V8 verknüpft bestehende Claims datenbewusst mit Policen, V9 entfernt die nicht mehr benötigte `customer_number`-Spalte und V10 ergänzt synthetische Demo-Daten. Angewandte Migrationen V1 bis V10 sind unveränderliche Historie.

Vorsichtiges Vorgehen:

1. Bestehende Migrationen `V1` bis `V10` nicht umbenennen oder nachträglich bearbeiten.
2. Neue Schemaänderungen nur unter einer noch freien Flyway-Version hinzufügen.
3. Alte Claims nur anhand der Kundennummer automatisch verbinden, wenn genau eine Police existiert. Unbekannte, mehrdeutige oder widersprüchliche Daten müssen die Migration vor Änderungen an den Claim-Daten stoppen.
4. Historisches `occurred_on` kann aus `created_at` abgeleitet werden, ist aber als Näherung zu dokumentieren.
5. Vor dem produktiven Einsatz eine frische und eine repräsentativ befüllte PostgreSQL-Datenbank testen.

Abnahmekriterien:

- Alle Flyway-Versionen sind eindeutig; es gibt keinen Validatorfehler.
- `V8` läuft nur bei eindeutigem Kunden-zu-Police-Mapping durch und stoppt mehrdeutige, fehlende oder widersprüchliche Altdaten ohne sie zu löschen.
- Legacy-Claim-Typen werden nur über bekannte Zuordnungen konvertiert; unbekannte Typen stoppen die Migration.
- V9 entfernt `customer_number` erst nach dem erfolgreichen Backfill der Claim-Policy-Zuordnung.
- Eine frische PostgreSQL-Datenbank lässt sich vollständig migrieren.
- `spring.jpa.hibernate.ddl-auto=validate` bestätigt das finale Schema.
- Wo bestehende Daten relevant sind, funktioniert der Upgrade-Pfad von einer repräsentativen bestehenden Schemahistorie ohne Datenverlust.
- Eine Integrationstest- oder reproduzierbare Prüfroute belegt die Migration mit PostgreSQL.
- H2-Seed, JPA-Entities und SQL-Schema widersprechen sich nicht bei den benötigten Spalten und Constraints.

R-02 – Entity-, Seed- und Schemaabgleich

Priorität: P1 – zusammen mit R-01 **Status:** Implementiert; PostgreSQL-Integrationstest noch offen

Claim-Entity und H2-Seed verwenden `occurred_on` und `updated_at`. V8 ergänzt diese Spalten für PostgreSQL; V9 entfernt `customer_number` nach dem Backfill. Die Migrationen sind implementiert, ein Integrationstest der vollständigen V1–V10-Historie mit PostgreSQL ist jedoch noch nicht in der Standardtestsuite nachgewiesen.

Abnahmekriterien: H2 und PostgreSQL besitzen dieselben fachlich erforderlichen Spalten, Nullability und Constraints; Claim-Anlage, Abruf und Änderung funktionieren in beiden vorgesehenen Profilen. Unterschiede zwischen DB-Herstellern werden ausdrücklich begründet und getestet.

R-03 – Echte Authentifizierung und Zugriff pro Kunde

Priorität: P0 – vor Einsatz mit echten Benutzern/Kundendaten **Status:** Offen – Anforderungen zuerst festlegen **Betroffene Dateien:** `shared/security/SecurityConfig.java`, Controller in Claim/Customer/Policy/Billing, Frontend-API und UI

Im erfassten Stand sind Benutzer In-Memory konfiguriert und Passwörter Beispieldaten. Die Claim-Routen prüfen Rollen, aber eine Verknüpfung der angemeldeten CUSTOMER-Identität mit genau den eigenen Policen/Claims ist nicht erkennbar. Frontend-API-Aufrufe richten keine Anmeldung ein.

Vor Implementierung entscheiden: Wer sind Kundinnen/Kunden und interne Mitarbeitende? Gibt es externe Identitätsverwaltung? Welche Rolle darf welche Aktion und welche einzelnen Datensätze sehen?

Abnahmekriterien: Keine fest codierten produktiven Zugangsdaten; sichere Konfiguration der Identität; dokumentierte Rollen-/Datensatzmatrix; serverseitige Autorisierung pro Ressource; Tests für erlaubte und verweigerte Zugriffe einschließlich zweier unterschiedlicher Kunden; Frontend überträgt Anmeldeinformationen sicher und behandelt Ablauf/Fehler nachvollziehbar.

R-04 – Fehlervertrag und API-Validierung

Priorität: P2 – vor großem Frontend-Ausbau **Status:** Offen – erst verstehen

Der globale Fehlerhandler deckt Not-found- und ausgewählte Domain-/State- Fehler ab. Validierungs- und Security-Antworten folgen möglicherweise anderen Antwortformaten. Frontend-Typen sind im erfassten Stand nicht vollständig mit den Backend-Responses abgestimmt.

Abnahmekriterien: Einheitlicher, dokumentierter Fehlervertrag (zum Beispiel Problem Details); korrekte HTTP-Statuscodes; dokumentierte API-Schemas; Frontendtypen stimmen mit tatsächlichen Responses überein; Tests für ungültige Requests, fehlende Ressourcen sowie 401/403.

R-05 – Billing- und Statusabläufe vollständig testen

Priorität: P1 – vor produktiver fachlicher Nutzung **Status:** Offen – erst verstehen

Betroffene Abläufe: Policy-Ausstellung → `PolicyIssuedEvent` → genau eine Rechnung; Claim `RECEIVED` → `IN_REVIEW` → `APPROVED` → genau eine Auszahlung. Auch Ablehnung, ungültige Übergänge, fehlende/gekündigte Police, Schadendatum außerhalb des Vertrags und Beträge über dem Deckungslimit prüfen.

Abnahmekriterien: Fachliche Erfolgspfade und relevante Ablehnungsfälle sind durch Service-/Modul-/Integrationstests abgedeckt; eine Eventwiederholung erzeugt keine doppelten Invoice-/Payout-Datensätze; Verhalten bei fehlgeschlagener Folgeverarbeitung ist definiert.

R-06 – Erforderliche Event-Zuverlässigkeit festlegen

Priorität: P1 bei verpflichtender Rechnung/Auszahlung; sonst nach Bedarf **Status:** Zurückgestellt bis Fehlertoleranz entschieden ist **Betroffene Dateien:** Modulith-Konfiguration, Event-Listener, Billing-Repositories und ADR `docs/adr/0002-spring-modulith-events-outbox.md`

Domain Events sind im Code implementiert; daraus allein folgt keine garantierte dauerhafte Wiederaufnahme nach Prozessausfall. Ein Outbox-Mechanismus oder externer Broker ist nicht vorsorglich einzuführen.

Vor Implementierung entscheiden: Darf Rechnung/Auszahlung unmittelbar nach dem Vorgang verzögert sein? Was muss bei App-/DB-Absturz garantiert werden? Welche Wiederholungs-, Dead-Letter- und Betriebsüberwachungsregeln sind erforderlich?

Abnahmekriterien: Entscheidung samt Alternativen/Risiken in einem ADR; Listener sind wiederholbar/idempotent; Fehler und Rückstände sind erkennbar; bei verpflichtender Dauerhaftigkeit belegen Fehler-/Restart-Tests die Wiederaufnahme ohne verlorenes oder doppeltes Geschäftsergebnis.

R-07 – Frontend-Anmelde- und Fachabläufe

Priorität: P2 – nach fachlicher/API-Klärung **Status:** Offen – erst verstehen **Betroffene Dateien:** `frontend/src/App.tsx`, `frontend/src/api/`, `frontend/src/features/`, Backend-Controller und Sicherheitsregeln

Die aktuelle UI bietet Listenansichten für Claims und Policen. Benötigte Funktionalität hängt von der geklärten Produktanforderung ab.

Abnahmekriterien: Unterstützte Benutzerabläufe sind konkret beschrieben; Login, Laden, Schreibaktionen, Validierungsfeedback und 401/403 sind durchgängig behandelt; keine Oberfläche bietet eine Operation an, die die Identität nicht sicher autorisieren kann; Frontend-Build und relevante Tests laufen.

R-08 – Repository-Schichten und Modulregeln vereinheitlichen

Priorität: P3 – schrittweise, wenn konkrete Änderung es rechtfertigt **Status:** Erledigt am 28.10.2026

Alle Fachmodule verwenden Spring-Data-Repositories direkt im `domain`-Paket. Nicht verwendete Repository-Ports und Weiterleitungsadapter wurden aus Customer, Product und Billing entfernt. Die Claim-Anwendungslogik liegt im `application`-Paket; leere Platzhalterordner und ungenutzte Verträge/Events sind ebenfalls bereinigt. Der Modulith-Test sichert die Modulgrenzen weiterhin ab.

Abnahmekriterien erfüllt: Entfernte Ports/Adapter und Verträge haben keine aktiven Aufrufer; Modulith- und Backend-Tests bestehen. ADR 0003 dokumentiert jetzt die direkte Repository-Struktur und deren bewusste Abwägungen.

R-09 – Betriebsbereitschaft und Konfigurationsschutz

Priorität: P2 vor gemeinsamem/stabilen Deployment; P0 für echte Geheimnisse **Status:** Teilweise umgesetzt – lokale und produktive Betriebsdetails weiter prüfen **Betroffene Dateien:** `docker-compose.yml`, `Dockerfile`, `application*.yml`, CI-Workflow

Das lokale `docker-compose.yml` wartet nur auf `service_started`; dies bestätigt nicht, dass PostgreSQL Verbindungen annimmt. Das separate `docker-compose.prod.yml` wartet auf einen PostgreSQL-Healthcheck und startet Backend und Caddy. Der Deploy-Workflow baut und veröffentlicht Backend- und Caddy-Images. Die

Security-Konfiguration verwendet weiterhin In-Memory- Demo-Benutzer und ist keine produktive Identitätsverwaltung.

Abnahmekriterien: Geheimnisse stammen aus geeigneter Laufzeitkonfiguration und nicht aus Produktions-Defaults; PostgreSQL-Bereitschaft wird zuverlässig behandelt; Health-/Readiness-Verhalten ist geklärt; Logs und Fehler erlauben Fehlersuche ohne sensible Daten; Build und Bereitstellung sind reproduzierbar; Frontend-Bereitstellung ist der tatsächlichen Zielumgebung entsprechend dokumentiert.

R-10 – Frontend- und Backend-Dokumentation synchron halten

Priorität: P3 – fortlaufend **Status:** In Arbeit – Stand 28.10.2026 aktualisiert

Abnahmekriterien: Architekturunterlagen beschreiben den tatsächlich implementierten Stand; Entscheidungen stehen in ADRs, wenn es mehrere bedeutsame Optionen gibt; API-Beschreibung passt zu den Endpunkten; PDFs sind aus der aktuellen Markdown-Fassung aktualisiert.

5. Was bewusst noch nicht auf der To-do-Liste steht

Diese Maßnahmen sind **keine automatischen nächsten Schritte**:

- die Anwendung in Microservices zerlegen,
- PostgreSQL durch MariaDB ersetzen,
- Hibernate/JPA vollständig gegen JDBC oder jOOQ austauschen,
- einen Message Broker wie Kafka einführen,
- eine Outbox allein wegen des Schlagworts „Enterprise“ hinzufügen,
- alle Module gleichzeitig auf eine einheitliche Architektur umschreiben,
- unbenutzte Libraries entfernen, bevor ihr Einsatz geprüft wurde,
- Frontend-Funktionen hinzufügen, bevor Rollen und Fachabläufe geklärt sind.

Eine dieser Maßnahmen kann später richtig sein. Zuerst muss ein konkretes Problem oder eine konkrete Anforderung zeigen, dass sich die zusätzlichen Kosten lohnen.

6. Änderungshistorie

Datum	Änderung
2026-09-28	Roadmap und Kommunikationsleitplanken erfasst; sichere V8-Backfill-Regel für Claims ergänzt
2026-10-28	Vereinheitlichte Paketstruktur, Migrationen bis V10 sowie lokale/produktive Bereitstellung aktualisiert

7. Verwandte Dokumente und Codepfade

- Ausführliche technische Architekturdokumentation
- Systemkontext-Diagramm
- Modul- und Schnittstellendiagramm
- Claim-zu-Auszahlung-Sequenz
- Policy-zu-Rechnung-Sequenz
- Kurzübersicht der Modularchitektur
- ADR: modularer Monolith

- ADR: Spring-Modulith-Events und Outbox
- ADR: Persistenzzugriff im Fachmodul
- Flyway-Migrationen
- Modul-/Architekturtests